Task-Aware Progressive Retrieval Attention: Virtualizing Interleaved Agent Context by Execution Scope

PRA Research Program

August 2026

Abstract

Long-running agents interleave records from several logical tasks inside one physical session. Session-wide retrieval therefore confuses physical availability with execution relevance. We introduce task-aware Progressive Retrieval Attention (PRA), which uses harness-authoritative task identity and workflow relations to restrict the retrieval space before ordinary PRA discovery and bounded materialization. In a frozen production native-Q/K evaluation over 15 typed-record cases, structural task scope raises complete evidence availability from 33.3% to 93.3%, removes 75.0% cross-task contamination, and reduces requested native tokens from 234.3 to 93.5. Exposing the same selected exact records visibly raises frozen-model answer accuracy from 13.3% to 66.7%. The advantage grows with semantic task confusability and persists on dependency workflows, although joins remain constrained by the physical selection budget. Validated JSON and Markdown acquisition exactly recover explicit task graphs; online-only acquisition is weaker, while hybrid mutation repairs the held-out join relation. A secondary native-consumption diagnostic identifies a query-position bug and reaches 60.0% with corrected all-layer memory, versus a 73.3% visible ceiling. These results support typed task structure as a first-class PRA scope signal. They do not establish latent task planning, sparse native-memory integration, or serving-scale efficiency.

EXPERIMENTALLY FROZEN / READY FOR EXTERNAL REVIEW

1 Introduction

Agent sessions are not necessarily single semantic episodes. A coding assistant may pause a database migration, inspect an unrelated test failure, resume the migration, and then consume a result produced by an earlier subtask. Its physical history is one token stream, but its execution state is a set of interleaved tasks and relations. Under a fixed retrieval budget, a session-wide router can spend capacity on recent or lexically similar records that belong to the wrong task.

We study a narrow claim: *explicit typed task state and workflow relations can serve as a first-class context-scope signal for PRA*. The harness first determines which task records are admissible. PRA then ranks within that partition, and the inherited typed-record runtime chooses how much exact content to materialize. The task graph is not inferred by the router, and PRA does not become a workflow engine.

The paper makes four contributions. First, it defines task-local, structural, and adaptive candidate scopes over a replayable typed-record stream. Second, it isolates the mechanism in controlled interleaving and resumption studies. Third, it confirms the effect with a frozen native-Q/K router and frozen Qwen3-0.6B generation: structural scope produces more complete evidence with fewer requested native tokens than session-wide routing. Fourth, it evaluates practical acquisition paths and separates their transcription quality from the oracle scope result. A native-consumption bug ablation is reported as a secondary integration diagnostic rather than part of the central scope claim.

2 Task-Aware PRA Architecture

Let the physical session be $H = (r_1, \ldots, r_n)$, with typed records assigned to tasks in a workflow graph G_T. Ordinary retrieval estimates relevance from a query q over the full session. Task-aware PRA instead

conditions admission on the active task T:

$$P(r \mid q) \quad \longrightarrow \quad P(r \mid T, q, G_T).$$

For active task T, the candidate set is

$$C(T, q) = M(T) \cup S(T) \cup R(T, q),$$

where $M(T)$ contains mandatory control state, $S(T)$ is an explicit structural closure, and $R(T, q)$ is controlled widening when the policy permits it. The default closure is

$$S(T) = \{T, \text{Parent}(T), \text{UnresolvedDeps}(T), \text{Blockers}(T), \text{RequiredEvidence}(T)\}.$$

The PRA router scores only admitted records or chunks. The materializer then maps selected identities to compact text, exact backing spans, or native K/V under a separate budget.

Task-local scope. Admit only records owned by the active task. This provides strict isolation but cannot recover predecessor outputs required by joins.

Structural scope. Admit the active task and its explicit dependency/evidence closure. This is the primary policy: it preserves known cross-task evidence without reopening unrelated history.

Adaptive scope. Begin with structural scope, detect incomplete or inconsistent metadata, and widen through related tasks to the session. Adaptive scope is a recall safeguard; widening increases contamination and cost and is recorded in provenance.

Scope breadth and materialization depth are independent controls:

$$\pi = (B_{\text{scope}}, B_{\text{detail}}).$$

Task structure answers *where to search*; PRA answers *which records or chunks*; the typed-record runtime answers *how much exact state to expose*.

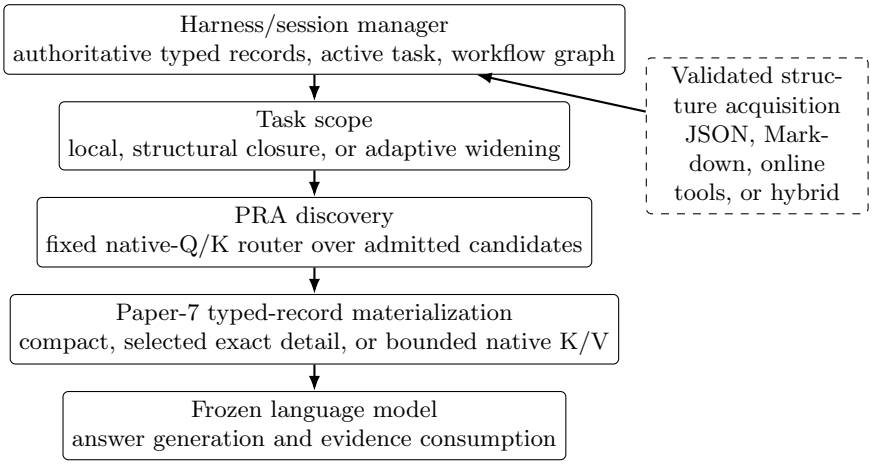

Figure 1: Task-aware PRA execution path. Structure acquisition proposes task metadata; the harness validates and owns it. Task scope precedes ordinary PRA discovery, and Paper-7 mechanisms retain responsibility for exact backing and progressive materialization.

3 Harness and Runtime Responsibility Boundary

The harness is authoritative for session and task identifiers, record provenance, task status, dependency edges, tool execution, workspace state, authorization, and validation of task transitions. Model-generated

operations are proposals until parsed, version-checked, and committed. The event stream is replayable and rejects stale versions, missing dependencies, self-links, and cycles.

The PRA runtime owns a reconstructible projection: candidate partitions, indexes, routing scores, selected identities, materialization state, and cold/host-encoded/device-hot working-set state. Durable sessions store typed records, task descriptors, and replay events, not model caches. This boundary matters causally: the primary experiment gives the runtime an oracle graph so that planning competence cannot be mistaken for a retrieval effect.

Paper 8 extends inherited typed records with task provenance but not a parallel memory format. If $R_7 = (id, \tau, P, A, V, B)$ denotes identity, type, policy/provenance, address views, visible view, and exact backing state, then

$$R_8 = (R_7, \Pi_T),$$

where Π_T records ownership, optional producer and consumers, relation type, and event sequence. Completed tasks are demoted from hot execution state to compact/index state; their exact backing remains addressable.

3.1 Negotiated task and record transport

Remote execution must not turn the harness projection back into one opaque user prompt. The agent retains an OpenAI-style conversational spine and supplies selected task, result, document, tool, and skill records as a separate logical resource stream. A canonical wire projection carries stable record identity, type, version, source fingerprint, task ownership/status, dependency provenance, named views, authorization scope, and tenant/session binding. It does not carry native K/V or physical cache identifiers.

The immediate endpoint advertises whether it accepts typed records, task metadata, session state, incremental messages, and resource deltas. With those features, the first task turn sends a full logical inventory; later turns send append-only messages and record operations. A transition from active to blocked, resumed, and completed changes the task-record version and emits an UPDATE even when its backing document is unchanged. Downstream storage can therefore preserve open-task dependencies and apply completion compaction without reconstructing task meaning from prose.

An ordinary endpoint receives a deterministic text rendering. A gateway backed by an ordinary engine can still accept typed records and own this downgrade; the agent should not discard task identity prematurely. On reconnect or engine restart, the agent repeats capability negotiation and resends the full resource inventory before returning to delta mode. Storage deduplication remains subordinate to authorization and never grants cross-session access.

4 Experimental Protocol

We order experiments by causal dependence.

Primary mechanism study. A deterministic model-free generator creates 80 workflows spanning parallel, linear, join, and DAG structures, with task counts $\{2, 4, 8, 16\}$, six records per task, and seeds $\{11, 23, 37, 53, 71\}$. Records are physically interleaved and use shared vocabulary. Scope policies receive the oracle task graph; within-scope lexical/recency ranking and record budgets are fixed.

Production confirmation. The production cohort contains 15 paired typed-record cases across independent, linear, join, resumption, and conflicting-state workflows at low, medium, and high semantic confusability. The router is the inherited native-Q/K hybrid path with top-$K = 8$. The frozen generator is Qwen/Qwen3-0.6B, revision `c1899de289a04d12100db370d81485cdf75e47ca`, with deterministic CUDA decoding. Only candidate admission changes between scope policies.

Acquisition study. Thirty explicit-structure workflows compare oracle, single-task, schema-constrained JSON, bounded Markdown, online task tools, and hybrid preflight-plus-online mutation. Graph quality and downstream routing utility are reported separately.

Secondary native diagnostic. A 15-case replay conditions on selected identities and varies visible versus native consumption, source/query positions, selected width, and consumer-layer depth. It diagnoses the model integration path; it is not additional evidence for scope selection.

Across studies we distinguish selection, exact materialization, model availability, and correct answer use. Primary routing metrics are required-record recall, useful precision, complete evidence availability, cross-task contamination, and requested unique native tokens. Generation metrics are exact answer accuracy and evidence use. Python latency counters are diagnostic because orchestration is not serving-optimized.

5 Oracle Task-Scope Mechanism

The controlled study asks whether scope alone changes allocation. At 16 parallel tasks, session-wide ranking reaches zero relevant-record recall and 1.00 contamination, whereas local, structural, and adaptive scope each reach 1.00 recall and zero contamination under the same two-record, 42-visible-token selection budget. Averaged across task counts, session-wide recall is 0.40 and contamination 0.80; all task-aware policies reach 1.00 recall and zero contamination in this construction.

Task-local isolation is not sufficient for dependent work. In an eight-input join it recovers one of eight required records and never completes the join. Structural closure admits all predecessors and reaches 1.00 recall and complete recovery, matching session scope without treating unrelated history as eligible. At 16 inputs, structural and session scope each recover only half the required records under the eight-record cap. Structure removes irrelevant competition; it does not enlarge physical capacity.

The resumption control reaches the same conclusion across time. After 128 intervening records, session-wide recall is zero with contamination one, whereas each task-aware policy retains recall one and contamination zero. Historical scaling, resumption, and scope–detail plots are preserved in Appendix C; they support the mechanism but are secondary to the production result.

6 Production-PRA Confirmation

Table 1 is the primary routing result. Session-wide routing often recovers one required chunk, producing high record-level recall, but complete answer evidence is available in only 33.3% of cases. Structural scope admits explicit dependencies, reaches 93.3% complete evidence availability, removes 75.0% contamination, and requests 93.5 rather than 234.3 native tokens per case. The token reduction is 60.1%. Task-local scope is cheaper but loses predecessor evidence.

Table 1: Primary production routing result over 15 paired cases. R is required-record recall, C cross-task contamination, E complete exact evidence availability, and K_V requested native tokens. Only candidate scope changes.

Scope	R	C	E	K_V
Session	0.889	0.750	0.333	234.3
Task local	0.700	0.000	0.600	55.7
Task structural	1.000	0.000	0.933	93.5

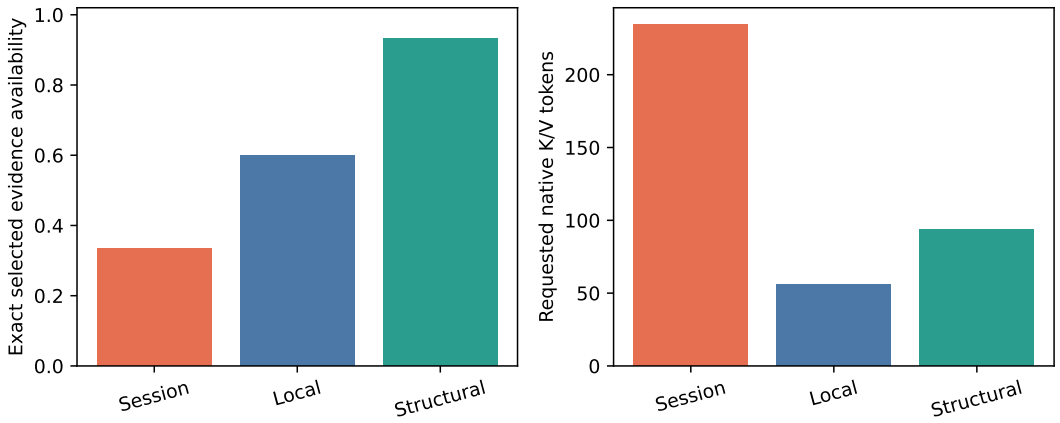

Figure 2: Production evidence economy from measured encoded chunks and index accounting. Structural scope makes complete evidence available more often while requesting fewer native tokens than session-wide routing.

The model-behavior controls in Table 2 show why complete scope matters beyond routing. When all exact records are serialized, full task scope reaches 73.3% accuracy versus 20.0% for the full session. With routed exact records serialized visibly, structural scope reaches 66.7% versus 13.3% for session routing. These are matched frozen-model comparisons. They show a positive task-scope effect when selected evidence is consumable; they do not imply that the original sparse native path was already functional.

Table 2: Primary frozen-model behavior. Full-context rows isolate context competition; routed-visible rows expose the selected exact records in the prompt.

Condition	Accuracy (%)	Evidence use (%)
FULL_SESSION	20.0	43.3
FULL_TASK_SCOPE	73.3	80.0
PRA_SESSION_VISIBLE	13.3	27.8
PRA_TASK_STRUCT_VISIBLE	66.7	74.4

7 Confusability, Joins, and Context Economy

The paired evidence advantage grows with semantic confusability: structural minus session evidence availability is 40.0, 60.0, and 80.0 percentage points at low, medium, and high confusability. Visible-answer gains are 40, 60, and 60 points. Each band contains only five heterogeneous cases, so Figure 3 is a bounded mechanism trend rather than a population scaling law.

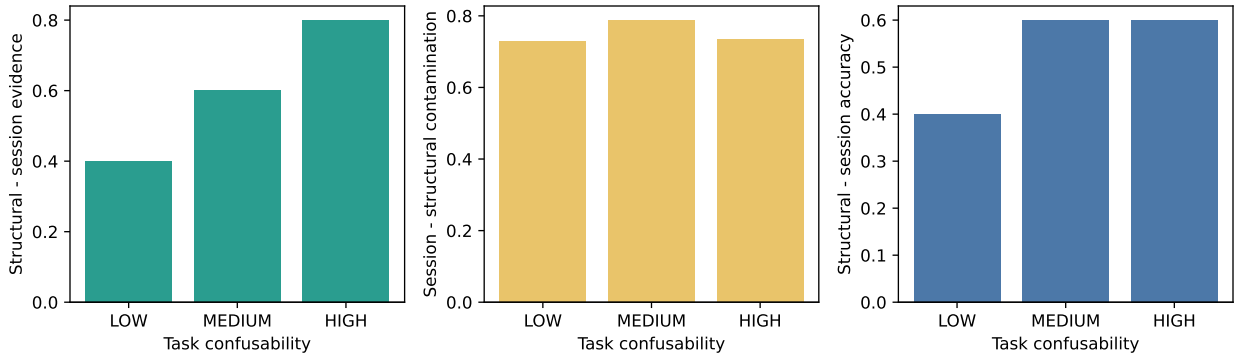

Figure 3: Paired structural-minus-session effects by task confusability. Exact evidence and visible answer accuracy improve as task states become harder to separate lexically. Intervals are case-bootstrap uncertainty over five heterogeneous cases per level.

Join fan-in exposes the boundary between logical scope and physical capacity. Fan-in 4 becomes retrieval-complete at encoded-ranking budget $K = 8$, fan-in 8 at $K = 16$, and fan-in 16 at $K = 32$. At fan-in 32, $K = 32$ recovers 0.744 of predecessor records because active-task and chunk overhead also consume positions. Frozen generation succeeds for fan-in 2 at $K \geq 4$ but fails for fan-in 4 and 8 even when retrieval is complete. Explicit structure identifies the required predecessors; budget and multi-record composition still determine whether they can be used.

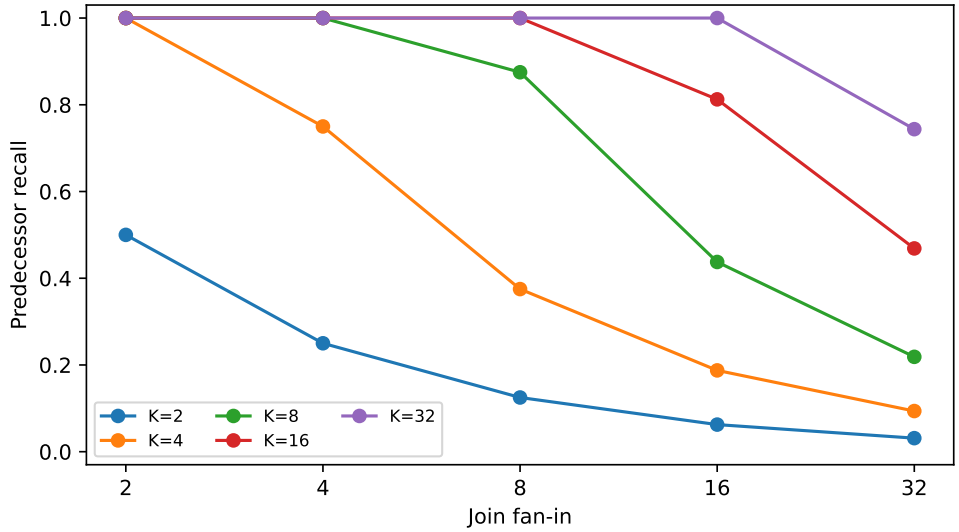

Figure 4: Predecessor recovery by join fan-in and encoded-chunk budget. Task structure removes unrelated competition but cannot override the selected-chunk budget.

Across five independent DAG families and five seeds, session and structural scope have equal aggregate required-record recall, 0.954, while structural scope removes all cross-task contamination. This matched result is important: structure narrows the candidate set without manufacturing a recall advantage when the graph already fits the budget.

8 Metadata Robustness and Adaptive Widening

Oracle structure isolates the mechanism but production metadata can be incomplete. Removing an active dependency lowers hard structural required-record recall to 70.0%; stale task tags lower it to 0.0%. Adaptive scope detects these inconsistencies and widens from structural to related and then session scope, restoring mean recall to 100.0% in the tested corruptions.

The recovery is deliberately not described as free. Every corrupted case admits all eight records after widening, increasing candidate cost and surrendering task isolation. Adaptive widening is therefore a conservative fallback with auditable provenance, not silent graph reconstruction. A deployment can cap widening, abstain, or request graph repair when the contamination risk is more costly than missing evidence.

9 Task-Structure Acquisition

The primary scope result uses oracle task identity and relations. We separately test whether explicit structure can enter the authoritative graph through practical interfaces. Requests expose task identifiers and dependencies; the frozen model transcribes them into schema-constrained JSON, a bounded Markdown grammar, or online create/link operations. Every proposal passes through the same parser, version checks, and DAG validation before it can mutate harness state.

Table 3: Acquisition over 30 explicit-structure workflows. Edge F1 measures validated graph recovery; downstream recall uses the generated graph for structural evidence selection.

Mode	Edge F1 (%)	Downstream recall (%)
Preflight JSON	100.0	100.0
Preflight Markdown	100.0	100.0
Online task tools	83.3	87.5
Hybrid	100.0	100.0

JSON and Markdown exactly recover the supplied graphs. Online-only acquisition reaches 83.3% edge F1 and 87.5% downstream recall: the five join outputs preserve dependency prose but omit structured `depends_on`. Hybrid preflight plus one validated execution-discovered link repairs the held-out join relation and returns to exact recovery. This experiment establishes explicit-structure transcription and controlled mutation. It does not show that a 0.6B model can infer latent tasks, resolve ambiguous task boundaries, or discover dependencies that are absent from the request and execution trace.

10 Native-Consumption Diagnostic

The original late-layer native path scored zero despite complete routed evidence. We therefore replayed the 15-case diagnostic cohort with a monotonic ladder: visible prefix, equivalent full native reference, full session, full task scope, required records, routed records, materialization widths, and consumer-layer depth.

The first defect occurred before routing. Source K/V retained positions $0, \ldots, M-1$, while the local query restarted at zero. Under rotary position embeddings this is not equivalent to an ordinary prefix. The corrected path begins the query after the longest source, tokenizes the rendered chat prompt once, and keeps request-owned memory alive through prefill and decode. One-reference checks preserve the top token and expected-token rank; all 28 native K/V recaptures have maximum reported numeric error 0.0000.

After correction, full task-scoped native memory consumed at all layers reaches 60.0%, versus the 73.3% visible ceiling in this diagnostic cohort. Full session-native memory remains at 0.0%. Required-record and routed-record full materialization both reach 60.0% at 80.2 unique tokens on average. Four sparse layers and the last layer alone remain at zero. The immediate remaining boundary is consumer depth and autoregressive composition, not first-token discovery: the first expected token has mean rank one even in the failing sparse conditions.

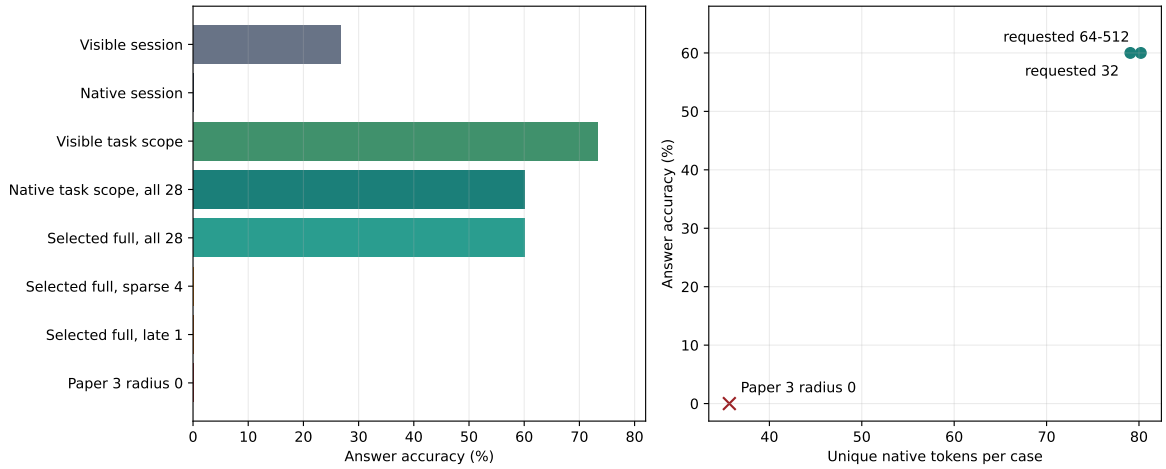

Figure 5: Secondary native-consumption diagnostic. Correct source/query geometry enables full all-layer task-scoped memory, while late sparse consumption remains insufficient. Requested widths are plotted at actual unique native tokens after interval union.

Canonical interval union removes 48.8 overlapping source positions per native condition on average; raw requests reach 2.0 times the unique count. This is bookkeeping rather than a new selection method. The frozen Paper-3 transfer uses its original 32-token parent, zero overlap, annotated-evidence oracle, radius zero, and MuSiQue all-layer band without Paper-8 retuning. It averages 35.7 tokens and scores 0.0% here. That transfer does not revise Paper 3's dataset-specific result; it shows that its QA-selected evidence core is insufficient for typed multi-record task outputs.

11 Related Work

Retrieval-augmented generation combines parametric generation with non-parametric document retrieval [1]. Task-aware PRA differs in the source of its first-stage signal: the harness already knows the active task and explicit workflow edges, so those relations constrain eligibility before semantic routing. The method complements rather than replaces content retrieval within an admitted task scope.

Memory-oriented agent systems such as MemGPT move information across context tiers to extend effective context [2]. ReAct interleaves model reasoning and environment actions [3]. Paper 8 addresses a different boundary: authoritative task state remains in the harness, while the model/runtime receives a replayable task projection that controls context admission. It neither proposes a general planner nor treats model-generated task operations as authoritative.

Prompt-compression methods such as LLMLingua reduce visible token cost by selecting or rewriting prompt content [4]. KV-cache policies such as H2O and StreamingLLM retain tokens according to attention importance or recency/sink structure [5, 6]. Task-aware PRA is orthogonal to those materialization and cache policies: it uses typed execution provenance to define the candidate scope, then delegates ranking and detail depth to PRA and the inherited record runtime.

12 Limitations

The production cohort contains 15 controlled cases from one generator and one 0.6B model. Five seeds are distributed across workflow types rather than providing five independent generations for every individual case. The confusability bands contain five heterogeneous examples each, so their monotonic trend is not a general scaling law.

Task identity and workflow relations are oracle inputs in the primary causal study. The acquisition study transcribes identifiers and dependencies that are stated explicitly; latent decomposition, ambiguous requests,

noisy tool traces, and long-horizon graph maintenance remain open. Metadata corruption is repaired only by widening to the full tested session, which restores recall while losing isolation.

Structural scope cannot materialize evidence that exceeds the physical budget. Large joins require larger budgets, staged consumption, or a learned composition mechanism. Even with complete retrieval, the frozen model fails some multi-record joins. Scope selection removes irrelevant competition but does not solve reasoning or composition.

Corrected full all-layer native consumption reaches 60.0%, 13.3 points below visible task scope, while late sparse consumption remains zero. This establishes a working full native path in the bounded diagnostic, not a production-quality sparse integration. The Paper-3 transfer is deliberately unretuned and should not be read as a cross-paper leaderboard comparison.

Finally, measurements come from one CUDA workstation and exhaustive Python orchestration. Token, byte, and lifecycle counters are real, but the study makes no serving-throughput, asymptotic scaling, or distributed-runtime claim. Authorization, privacy boundaries, adversarial task metadata, and multi-user isolation are outside the experiment.

The negotiated task transport is a product contract, not an additional task-scope accuracy result. Preserving typed metadata across the wire prevents a representational loss and supports lifecycle policy, but it does not show that a remote model plans better or that delta transport lowers TTFT. Those claims require matched TEXT/PRA execution and serving measurements with the same selected task records.

13 Conclusion

In an interleaved agent session, the physically available history is a poor default retrieval scope. Explicit typed task state and workflow relations give PRA a sharper first-stage signal: structural scope makes complete evidence available in 93.3% rather than 33.3% of production cases, removes measured cross-task contamination, and requests 93.5 rather than 234.3 native tokens. The same scope improves frozen-model behavior when selected exact evidence is consumable. Hard local filtering fails on dependencies; adaptive widening recovers from damaged metadata at the cost of isolation; and joins remain bounded by materialization and composition capacity. These results support task structure as a practical context-allocation primitive, with authority retained by the harness and latent planning left to future work.

A Typed Task Records and Lifecycle

The canonical harness projection is

$$\begin{aligned}\text{TaskState} = \{&id, version, status, description, constraints, parent,\\ &dependencies, blockers, evidence, outputs, result\_ref,\\ &created\_seq, updated\_seq\}.\end{aligned}$$

Status is one of `pending`, `active`, `blocked`, `completed`, or `cancelled`. A compact `result_ref` points to a typed result record rather than embedding a large summary in authoritative state. Events include activation, blocking, resumption, completion, and cancellation. Applying an event recomputes scope and may promote, demote, or invalidate reconstructible caches.

Cold, warm, and hot resumption are distinct: cold state has backing without encoding, warm state has host encoding but is inactive, and hot state is accelerator-resident. The implementation records promoted, demoted, and reused tokens; these counters do not claim measured PCIe transfer.

B Workflow Families, Controls, and Metrics

The workflow families are atomic; linear chains; independent parallel sets; forks; joins; and general DAGs. We record task count N_T, edge count N_E, critical-path depth D, maximum width W, join count J, and fork count F. Orthogonal controls vary task switches, resumption distance, semantic similarity, shared entities, records per task, dependency-result delay, completed history, and hot wrong-task state.

Quality metrics include relevant-record recall, useful precision, contamination, complete evidence availability, join completeness, answer accuracy, and evidence use. Systems metrics include requested and unique native tokens, serialized prompt tokens, encoded backing tokens, index bytes, resident detail-cache bytes, and indexing/query/routing/materialization time. Acquisition reports parse validity, edge precision/recall/F1, join recovery, and downstream evidence recall.

C Oracle Mechanism Details

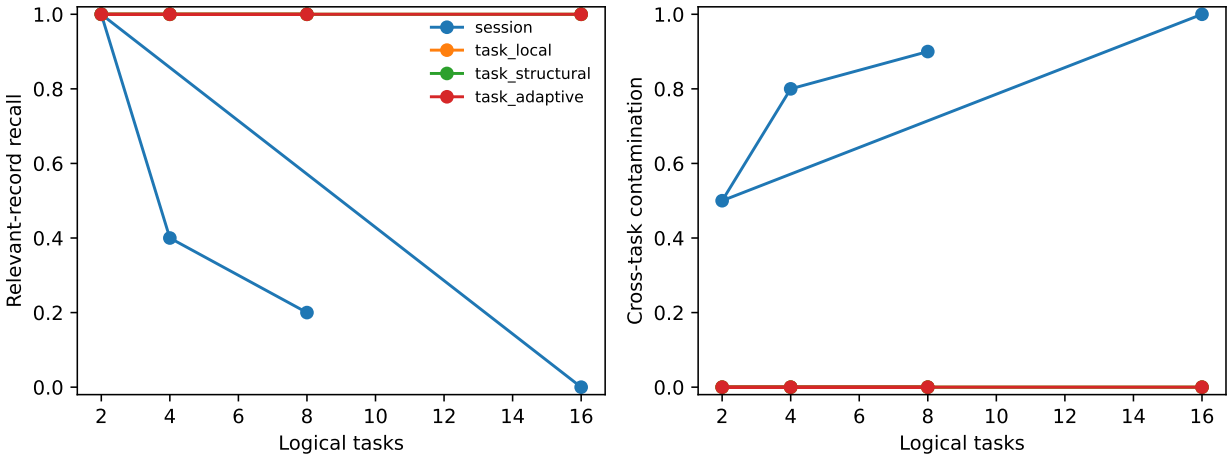

Figure 6: Controlled parallel-workflow scaling under a matched record budget. Session ranking increasingly selects recent, semantically similar wrong-task records; task-aware admission preserves active-task recall in this construction.

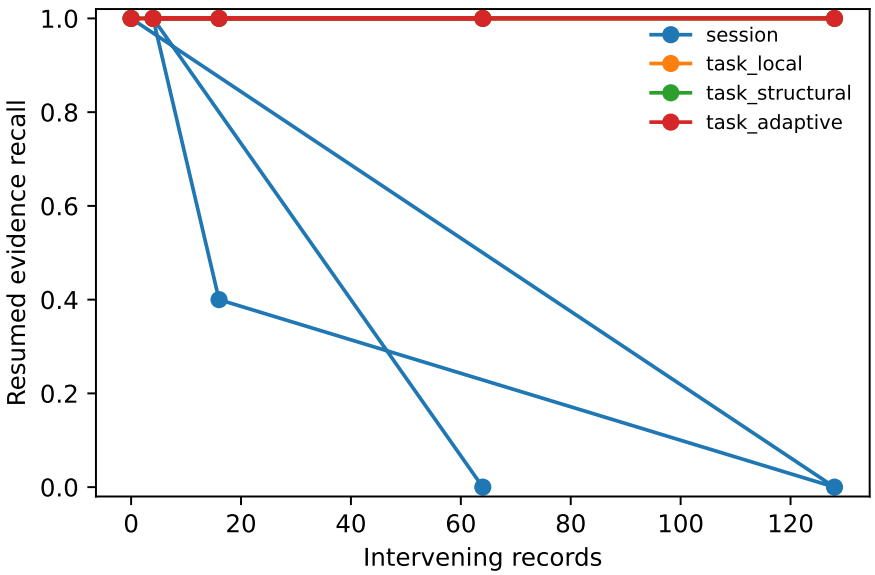

Figure 7: Oldest-task resumption after increasing intervening history. Task-aware policies retain the required record; session ranking is displaced by recent wrong-task evidence.

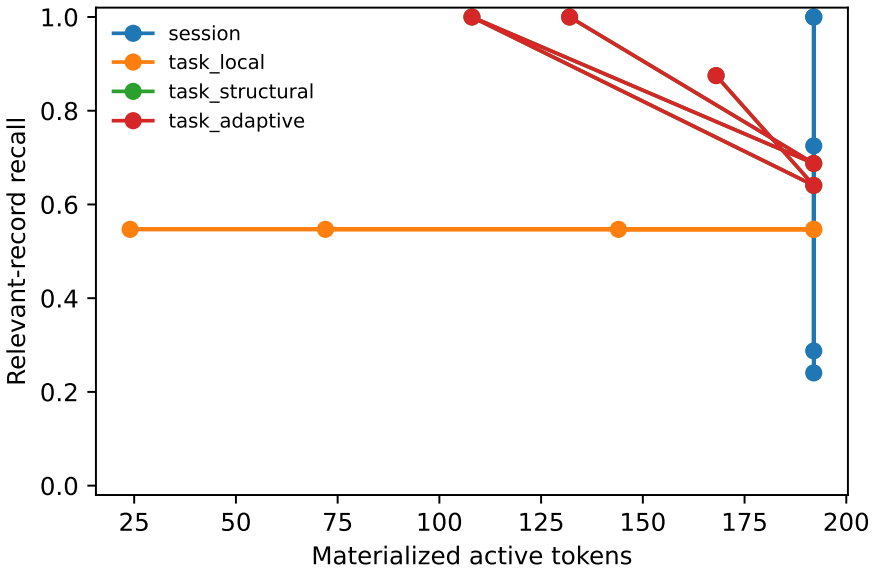

Figure 8: Historical scope–detail accounting under a 192-token allowance. Structural scope spends the same or fewer charged tokens than session scope while recovering more relevant records at deeper views. Costs are explicit accounting constants, not measured native transfer.

At selected detail, structural scope averages 0.875 recall with 168 charged tokens, versus 0.725 recall and 192 tokens for session scope. At native depth, the corresponding recalls are 0.641 and 0.241 at 192 charged tokens. These historical fixed costs validate policy accounting. The production experiment replaces them with measured encoded spans.

D Production Accounting and Replay

The production path records serialized prompt tokens, encoded backing-chunk tokens, requested and unique native positions, routing-index bytes, resident detail-cache bytes, and stage latency. Candidate admission returns record IDs; the inherited router indexes exact backing, ranks original chunks, and materializes the selected source-position union. Persistent session replay reconstructs admitted tasks, admitted records, and typed provenance for all 15 cases after model caches are discarded. Parent and dependency edges participate in cycle validation, including two-node and longer cycles.

The model controls distinguish four events: a required record is selected; its exact backing is materialized; the model can attend to that evidence; and the generated answer uses it correctly. The visible diagnostic serializes the same selected exact detail and is therefore a consumption localization control, not a separate retrieval method.

E Native Diagnostic Details

The corrected query starts after the longest independently positioned source. At FP16, equivalent prefix and native-reference runs have maximum and mean absolute logit differences 0.0391 and 0.0070. Native memory remains active through every recorded prefill and decode call in 100.0% of conditions. Canonical interval union never merges distinct records.

Materialization width is saturated by short records: requested widths 32 through 512 all reach 60.0% when consumed at all layers. The selected records average 80.2 unique tokens. Late sparse layers 11, 17, 23, and 27 score 0.0%; layer 27 alone scores 0.0%. The result isolates consumer depth, but does not determine whether fine-tuning, repeated injection, or another interface is sufficient.

F Implementation and Reproduction

`TaskState`, `TaskEvent`, graph validation, replay, and provenance live in `src/pra_hf/task_context.py`. Scope policies and working-set accounting live in `src/pra_hf/task_scope.py`. Abstract, in-memory, and local atomic session services live in `src/pra_hf/session_service.py`. Deterministic preflight parsing is in `src/pra_hf/task_planning.py`. Workflow generators are in `src/data/task_workflows.py` and `src/data/task_production_cases.py`.

The frozen manuscript is on branch `research/paper8-tasks`. Exact commands, artifact paths, model revision, and the manuscript-source commit are recorded in `docs/papers/paper8_tasks/README.md`. Generated metrics are under `docs/papers/shared/results/paper8_tasks/`; figures are under `docs/papers/shared/figures/paper8_tasks/`.

G Hypotheses, Falsification, and Program Boundary

The evidence supports the following bounded statements: interleaving can displace relevant records under session scope; structural scope preserves explicit dependencies better than task-local filtering; task scope reduces contamination under matched routing; and validated explicit-structure acquisition can recover the oracle graph in constrained formats. The evidence does not establish asymptotic sublinear working sets, reliable latent decomposition, sparse frozen native integration, or serving speed.

The thesis should be weakened if session routing matches structural scope under stronger interleaving at matched budget; structural closure routinely widens to near-global scope; metadata repair costs exceed its benefit; completion demotion harms resumption; generated task graphs fail to improve downstream selection; or gains occur only when synthetic labels leak into queries.

Paper 8 remains single-session. Separate-session subagent context trees and cross-session memory are reserved for later work. The paper does not claim invention of task lists, workflow DAGs, decomposition prompting, or task planning.

References

[1] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.

[2] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. arXiv:2310.08560, 2023.

[3] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023.

[4] Huiqiang Jiang, Qianhui Wu, Chin-Yew Lin, Yuqing Yang, and Lili Qiu. LLMLingua: Compressing prompts for accelerated inference of large language models. In *Proceedings of EMNLP*, pages 13358–13376, 2023.

[5] Zhenyu Zhang, Ying Sheng, Tianyi Zhou, Tianlong Chen, Lianmin Zheng, Ruisi Cai, Zhao Song, Yuandong Tian, Christopher Ré, Clark Barrett, Zhangyang Wang, and Beidi Chen. H2O: Heavy-hitter oracle for efficient generative inference of large language models. In *Advances in Neural Information Processing Systems*, 2023.

[6] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, 2024.